

Web Component

Costume Elements

- Costume Elements нь өөрийн шинэ HTML tag хийх боломжийг олгодог
- Үүний тулд тухайн элементийн нэртэй `class` үүсгэж үүсгэхдээ `extends HTMLElement` гээд `attribute`-ууд болон `method`-уудыг зааж өгнө
- Class-ийг тодорхойлсны дараа тухайн элементийг `customElements.define("ашиглах tag-ийн нэр", Үүсэгсэн Классын нэр);` -ээр бүртгэнэ.
- Ингээд html дээрээ tag-аа ашиглана.
- Хэрэв tag-аа тусдаа .js файл болгосн бол заавал классын урд `export default` заалгаж, ашиглах газраа `import 'тухайн файлын path'` -ийг зааж өгнө. Үгүй бол html дотроо `<script>`

HTML дээр script tag-тай бичсэн жишээ:

```
<my-greeting></my-greeting>

<script>
class MyGreeting extends HTMLElement {
  connectedCallback() {
    this.innerHTML = "<p>Сайн байна уу!</p>";
  }
}

customElements.define("my-greeting", MyGreeting);
</script>
```

Өөр file дээр costume element бичиж export,import хийсэн жишээ:

```
export class ProductCard extends HTMLElement {  
  constructor() {  
    super();  
  }  
  
  connectedCallback() {  
    this.render();  
    this.setupListeners();  
  }  
}
```

```
// Register component  
customElements.define('product-card', ProductCard);
```

```
import './product-card.js';  
import './product-card.css';
```

Shadow DOM

- Shadow DOM гэдэг нь HTML element-ийн дотор тусгаарлагдсан DOM tree үүсгэдэг энэ дахин ашиглах боломжийг дэмждэг
- Өөрөөр хэлбэл component-ийн доторх HTML + CSS-г гадна орчноос тусгаарлаж хамгаалдаг.
- Ингэснээр гаднах CSS дотор нөлөөлөхгүй ба дотор CSS гадна талд нөлөөлөхгүй.

Сул тал:

- Гаднаас style хийх буюу override хийхэд хэцүү
- Nested shadow root олон бол debug хийхэд төвөгтэй
- Зарим хуучин tool Shadow DOM дэмждэггүй

Shadow Root гэж юу вэ?

- Доорх жишээ-ний method нь element дээр shadow root үүсэгдэг

```
this.attachShadow({ mode: "open" });
```

- Хэрэв mode: open бол гаднаас хандаж болно.
- mode: closed бол гаднаас хандаж болохгүй.

Why use mode? you may ask

- Open mode-ийг ихэвчлэн debug хийх, test хийх, component customize хийхэд хэрэгтэй байдаг
- Харин closed mode-ийг component-ийн дотоод бүтцийг нуух, гаднаас хэн нэгэн DOM өөрчлөөд component эвдэхээс сэргийлэх буюу encapsulate хийхийн тулд ашигладаг.

Shadow DOM-гүй

```
<style>
  p {
    color: red;
  }
</style>
<div>
  <p>Hello</p>
</div>
```

Hello

Shadow DOM-той

```
<style>
  p {
    color: red;
  }
</style>
<div>
  <p>Hello</p>
</div>
<my-box></my-box>
<script>
class MyBox extends HTMLElement {
  constructor() {
    super();

    const shadow = this.attachShadow({ mode: "open" });

    shadow.innerHTML = `
      <style>
        p {
          color: blue;
        }
      </style>

      <p>Hello</p>
    `;
  }
}
```

Hello

Hello

HTML template

- `<template>` нь HTML дээр бичигдсэн үед харагдахгүй ч дараа нь javascript-ээр хуулж ашиглана.
- `<template>` ашиглах гол шалтгаан нь HTML бүтэцээ дахин ашиглах, цэвэр зохион байгуулах, мөн DOM-д хэрэгтэй үедээ л оруулах боломжтой болгодог.
- Нэг HTML загварыг олон удаа clone хийж ашиглаж болно.
- DOM дээр шууд олон element үүсгэхээс илүү template clone хийх нь хурдан, цэвэр байдаг.

Жишээ

```
<div class="card">  
  <h2>template baihgui</h2>  
  <p></p>  
</div>
```



template baihgui

```
<template id="card-template">  
  <div class="card">  
    <h2></h2>  
    <p></p>  
  </div>  
</template>  
  
<script>  
const template = document.getElementById("card-template");  
  
const clone = template.content.cloneNode(true);  
clone.querySelector("h2").textContent = "Гарчиг";  
clone.querySelector("p").textContent = "Энэ бол card-ийн текст";  
  
document.body.appendChild(clone);  
</script>
```



Гарчиг

Энэ бол card-ийн текст

What is <slot>?

- <slot> нь Shadow DOM дотор хэрэглэж болдог гадны HTML content-ийг insert хийхэд ашиглагддаг tag юм.
- Ихэнхдээ costum element-ийн дотор байгаа сул tag-уудыг javascript-ийн Shadow DOM –оор дамжуулан costum element-дотор шууд оруулдаг.
- Мөн нэртэй slot гэж байдаг ба үүнийг хэрэглэхдээ:
- Html дотор slot="нэр онооно" ба script дотроо <slot name="оноосон нэр">-ийг бичнэ.

```
<user-card name="Bat">  
  <button>Follow</button>  
  <p slot="username">Neguun</p>  
</user-card>
```

```
<!-- SLOT HERE -->  
<slot></slot>  
<slot name="username"></slot>  
</div>
```

- Энэ жишээ дээр user-card costum element дотор button гэсэн элемент сул байна
- Харин user-card-ийн component дотроо Shadow DOM ашигласан байгаа ба innerHTML дотроо <slot> tag бичсэнээр html дээр бичсэн button тухайн component дотор slot буюу байрлалд очиж бичигдэнэ.

```
<user-card name="Bat">
  <button>Follow</button>
</user-card>

<script>
class UserCard extends HTMLElement {
  constructor() {
    super();

    const shadow = this.attachShadow({ mode: "open" });

    shadow.innerHTML = `
      <style>
        .card {
          border: 1px solid black;
          padding: 10px;
        }
      </style>

      <div class="card">
        <h2>${this.getAttribute("name")}</h2>

        <!-- SLOT HERE -->
        <slot></slot>
      </div>
    `;
  }
}

customElements.define("user-card", UserCard);
```

ConnectedCallback()

- Тухайн element HTML page-д орох үед автоматаар хийгддэг үйлдлийг тодорхойлоход ашиглагддаг

DisconnectedCallback()

- Тухайн component HTML-ээс устах үед хийгддэг үйлдлийг тодорхойлдог.

```
class LiveTimer extends HTMLElement {  
  connectedCallback() {  
    // Starts the timer when the element hits the screen  
    this.timerId = setInterval(() => {  
      console.log("Секунд өнгөрлөө (A second passed)");  
    }, 1000);  
  }  
  
  disconnectedCallback() {  
    // Stops the timer when the element is removed  
    clearInterval(this.timerId);  
    console.log("Таймер зогслоо (Timer stopped).");  
  }  
}
```